

FitAI

Application d'essayage virtuel de vêtements par Intelligence Artificielle

Loïc Botsy · Titre Professionnel Concepteur Développeur d'Applications (Niveau 6)

StyleShop SAS · Juin 2026

StyleShop SAS — L'entreprise d'accueil

Qui sont-ils ?

- **Startup française** fondée en 2022, Paris 11ème
- **Activité** : Marketplace de vêtements de mode en ligne
- **14 collaborateurs** — équipe tech de 6 personnes
- **CA 2025** : 1,2 M€ · 32 000 utilisateurs actifs
- **Stack** : Django (back) · Flutter (mobile) · AWS (infra)

Mon rôle

Stagiaire CDA — développeur full-stack

Sous la supervision de **Thomas Dupont** (CTO)

Sophie Mercier — PDG

└─ Thomas Dupont — CTO ← tuteur

└─ 2× Back-end (Django)

└─ 1× Front-end (React)

└─ 1× DevOps (AWS)

└─ Loïc Botsy (stagiaire) ←

└─ Juliette Arnaud — Produit

└─ Product Manager

└─ UX Designer

└─ Camille Renard — Commercial

Expression des besoins — Le problème business

Le problème identifié

35 % des achats de vêtements en ligne sont retournés.
Raison principale : *"Ce n'est pas ce que j'imaginais sur moi."*

Impact business :

- Coûts logistiques retours : ~15 €/colis
- Insatisfaction client · churn
- Empreinte carbone élevée

La solution FitAI

L'utilisateur **visualise le vêtement porté sur sa propre photo** avant d'acheter.

Besoins fonctionnels prioritaires

ID	Fonctionnalité	Priorité
F1	Authentification sécurisée (JWT)	● Haute
F2	Upload 2 photos + description	● Haute
F3	Génération IA (30-90 s)	● Haute
F4	Affichage sécurisé du résultat	● Haute
F5	Historique des essayages	● Moyenne
F6	Téléchargement / Partage	● Moyenne

Architecture technique — Choix et justifications



Pourquoi ces technologies ?

Choix	Alternative écartée	Raison
Flutter	React Native	Déjà utilisé chez StyleShop
Django 5	FastAPI	ORM + SimpleJWT intégrés
PostgreSQL	MySQL	UUID natif + ACID
JWT	Sessions	Mobile stateless
IDM-VTON	API payante	Budget startup (gratuit)

Gestion de projet — Méthode Agile Scrum

6 sprints × 2 semaines

Sprint	Focus	Dates
S0	Cadrage, maquettes, setup	7–18 avr.
S1	Auth backend (JWT)	21 avr.–2 mai
S2	API TryOn + HuggingFace	5–16 mai
S3	Frontend Flutter	19–30 mai
S4	Sécurité + Historique	2–13 juin
S5	Tests + Documentation	16–27 juin

Outils

- GitHub
- Jira
- Figma
- Postman
- Slack

Kanban Sprint 3 (extrait)

À faire	En cours	Terminé
HistoryScreen	ResultScreen	LoginScreen
	AuthInterceptor	RegisterScreen
		TryOnScreen
		ImagePickerCard

Gestion des risques clés

- ⚠ Latence HuggingFace (30-90s)
→ Indicateur de chargement explicite
- ⚠ Cold start IDM-VTON (jusqu'à 90s)
→ Timeout 300s + HTTP 502 propre

Interface Flutter — Authentification



[CAPTURE
D'ÉCRAN]

Écran de
Connexion

Champs username +
password
Bouton "Se connecter"
Lien "S'inscrire"

login_screen.dart — Logique de connexion

```
Future<void> _submit() async {  
  if (_formKey.currentState!.validate()) {  
    try {  
      // Délègue au AuthNotifier (Riverpod)  
      // → appel POST /api/auth/token/  
      await ref.read(authProvider.notifier)  
        .login(_usernameCtrl.text.trim(),  
              _passwordCtrl.text);  
  
      // Succès → GoRouter vers '/home'  
      if (mounted) context.go('/home');  
    } catch (e) {  
      // Erreur serveur → Snackbar rouge  
      ScaffoldMessenger.of(context).showSnackBar(  
        SnackBar(content: Text(e.toString()),  
                  backgroundColor: Theme.of(context)  
                    .colorScheme.error),  
      );  
    }  
  }  
}
```

Interface Flutter — Écran Try-On

 [CAPTURE] TryOnScreen videDeux cartes "Appuyez pour sélectionner"

 [CAPTURE] Chargement IA CircularProgressIndicator + "Génération en cours (30-60s)..."

Pattern Riverpod — TryOnNotifier

```
// L'UI écoute deux signaux :
// 1. errorMessage → Snackbar rouge
// 2. resultImageUrl → navigation '/result'
ref.listen<AsyncValue<TryOnState>>(
  tryOnProvider, (prev, next) {
    if (next.value?.errorMessage != null)
      ScaffoldMessenger.of(context)
        .showSnackBar(...);

    if (next.value?.resultImageUrl != null
      && prev?.value?.resultImageUrl == null)
      context.push('/result'); // GoRouter
  }
);

// AbsorbPointer : bloque TOUTES
// les interactions pendant la génération IA
return AbsorbPointer(
  absorbing: isLoading,
  child: /* Formulaire */ ...
);
```

Composant métier — TryOnService

Diagramme de séquence simplifié



En cas d'erreur HuggingFace → TryOnAPIException
 → HTTP 502 + request_id pour traçabilité

services.py — Appel Gradio

```

class TryOnService:
    SPACE_ID = "yisol/IDM-VTON"

    @classmethod
    def generate_tryon(cls, person_path,
                      garment_path, description):
        start = time.time()
        try:
            client = Client(cls.SPACE_ID,
                           httpx_kwargs={"timeout": 300})

            result = client.predict(
                {"background": handle_file(person_path),
                 "layers": [], "composite": None},
                handle_file(garment_path),
                description,
                True, False, 30, 42,
                api_name="/tryon"
            )
            logger.info(f"✅ {time.time()-start:.1f}s")
            return result[0] # chemin temporaire

        except Exception as exc:
            raise TryOnAPIException(str(exc))
  
```


Modèle Django & API REST

Endpoints exposés

Méthode	URL	Auth	Code
POST	/api/auth/register/	✗	201
POST	/api/auth/token/	✗	200
POST	/api/auth/token/refresh/	✗	200
GET	/api/auth/profile/	✓ JWT	200
POST	/api/tryon/	✓ JWT	201
GET	/api/tryon/	✓ JWT	200
GET	/api/tryon/{uuid}/	✓ JWT	200
GET	/media/{path}	✓ JWT	200

Isolation multi-tenant

```
def get_queryset(self):
    # Filtrage strict : chaque user
    # ne voit QUE ses propres requêtes
    return TryOnRequest.objects.filter(
        user=self.request.user
    ).order_by('created_at')
```

models.py — Clés de conception

```
class TryOnRequest(models.Model):
    class Status(models.TextChoices):
        PENDING = 'PENDING'
        PROCESSING = 'PROCESSING'
        COMPLETED = 'COMPLETED'
        FAILED = 'FAILED'

    # UUID : non-prédictible → résistance BOLA
    id = models.UUIDField(
        primary_key=True,
        default=uuid.uuid4,
        editable=False
    )
    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE # RGPD : cascade
    )
    # Validator MIME sur les images d'entrée
    person_image = models.ImageField(
        validators=[validate_image_file]
    )
    garment_image = models.ImageField(
        validators=[validate_image_file]
    )
    # result_image : pas de validator
    # → généré par le backend, pas l'user
    result_image = models.ImageField(
        blank=True, null=True
    )
    status = models.CharField(max_length=20)
    created_at = models.DateTimeField(auto_now_add=True)
```

Sécurité — Conformité OWASP Top 10

OWASP 2021	Statut	Mesure principale
A01 — Access Control	✓	<code>filter(user=request.user)</code> · UUID
A02 — Crypto Failures	✓	JWT HS256 · <code>FlutterSecureStorage</code>
A03 — Injection	✓	ORM Django (0 SQL brut)
A04 — Insecure Design	✓	Rate limiting 10/h · UUID
A05 — Misconfiguration	✓	<code>DEBUG=False</code> · <code>ALLOWED_HOSTS</code>
A07 — Auth Failures	✓	Access 15 min · Refresh 7 j
A08 — Integrity	✓	Validation MIME magic bytes
A09 — Logging	✓	Logger <code>tryon.services</code> INFO/ERROR
A10 — SSRF	✓	URL HuggingFace codée en dur

Focus — Validation MIME (A08)

```
# validators.py
# Résiste à l'extension spoofing :
# un PDF renommé en .jpg est détecté

def validate_image_file(file):
    if file.size > 10 * 1024 * 1024:
        raise ValidationError("Fichier > 10 Mo")

    # Lecture des magic bytes réels
    file.seek(0)
    header = file.read(2048)
    file.seek(0)

    kind = filetype.guess(header)
    mime = kind.mime if kind else 'unknown'

    if mime not in ['image/jpeg',
                   'image/png',
                   'image/webp']:
        raise ValidationError(
            f"Type invalide : {mime}"
        )
```

Jeu d'essai — 8 cas testés

N°	Scénario	Résultat	Statut
T01	Inscription valide	HTTP 201	✓
T02	Connexion JWT	HTTP 200 + tokens	✓
T03	Upload PDF renommé .jpg	HTTP 400 + message MIME	✓
T04	11ème requête en 1h	HTTP 429 + Retry-After	✓
T05	Isolation user1/user2	Liste vide pour user2	✓
T06	Accès fichier autre user	HTTP 403 Forbidden	✓
T07	Route sans token	HTTP 401	✓
T08	Token expiré (Flutter)	Refresh auto · 0 erreur visible	✓

8/8 — Taux de conformité 100 %



- ⚠ En production : Redis requis pour le rate limiting
- ⚠ Redirection auto vers login si refresh expiré (phase 2)

Veille sécurité — Sources & vulnérabilités identifiées

Sources consultées

- CVE Mitre
- OWASP News
- PyPI Advisories
- Django Security Blog
- CERT-FR
- GitHub Dependabot
- PortSwigger Research

CVE identifiées

CVE	Composant	Impact	Action
CVE-2024-56374	Django < 5.0.11	DoS	Version 5.0.14
CVE-2024-3116	pgAdmin < 8.6	RCE	Mise à jour

Action corrective majeure — Supply chain

Problème identifié :

- python-magic-bin (validation MIME initiale)
- abandonnée depuis 2023
- DLL libmagic v1.0.17 de 2009 (non patchée)
- Fork suspect sur PyPI avec même nom

```
# AVANT (risqué)
import magic
mime = magic.from_buffer(header, mime=True)

# APRÈS (corrigé)
import filetype # Pure Python, maintenu
kind = filetype.guess(header)
mime = kind.mime if kind else 'unknown'
```

- 0 dépendance binaire système
- Maintenu activement

Synthèse

✓ Satisfactions

- Architecture propre et évolutive
- Sécurité pensée dès la conception (OWASP 9/10)
- `AuthInterceptor` : refresh JWT transparent
- Validation MIME robuste (magic bytes)
- Suite de tests significative (20 tests)
- 8/8 cas de test conformes

⚠ Difficultés

- **Latence HuggingFace** (30-90s) : UX à retravailler
- **Format Gradio** d'IDM-VTON : dict `ImageEditor` non documenté → 2 jours de débogage
- **Permissions galerie**
iOS/Android : comportement différent par plateforme
- **python-magic-bin** : dépendance abandonnée découverte tardivement

🚀 Axes d'amélioration

- **Redis** pour le rate limiting multi-workers
- **File d'attente** (Celery) pour passer l'IA en asynchrone
- **Stockage S3** (AWS) à la place du système local
- **HTTPS + HSTS** pour le déploiement production
- **Redirection auto** vers login si refresh expiré
- **Tests iOS** (permissions spécifiques)

Merci pour votre attention

Questions ?

Loïc Botsy · loic.botsy@hotmail.com

Code source : dépôt GitHub privé StyleShop · MSP1_BL

Stack : Flutter 3 · Django 5 · PostgreSQL 15 · HuggingFace IDM-VTON